Supportive Edges, Radius-Stable Peeling and Rooted Yang–Lu Replacement: the Randić–Radius Conjecture for Graphs of Cyclomatic Number at most 8

with a structure theorem for minimal counterexamples in general

Keio

25 October 2026

Abstract

The Randić index of a graph G is $R(G) = \sum_{uv \in E(G)} (d_u d_v)^{-1/2}$. In 1988 Fajtlowicz's program Graffiti conjectured that $R(G) \geq \mathrm{rad}(G) - 1$ for every connected graph. In 2000 Caporossi and Hansen conjectured the stronger bound $R(G) \geq \mathrm{rad}(G)$ for every connected graph other than a path on an even number of vertices. Both conjectures are open. Known cases include trees, unicyclic and bicyclic graphs, chemical graphs (maximum degree at most four), cacti and graphs on at most ten vertices.

Status. This paper does *not* prove the Caporossi–Hansen conjecture in general. It proves it for every connected graph whose cyclomatic number $|E| - |V| + 1$ is at most 8, and it proves a structure theorem for minimal counterexamples. That theorem reduces the general conjecture to excluding two explicit kinds of local configuration.

The method rests on three tools.

1. An exact formula for the change of R when one edge is deleted. It isolates *supportive* edges, those whose deletion increases R.
2. A *radius-stable peeling lemma*: deleting any vertex set S such that each vertex of S has a neighbour outside S lowers the radius by at most one.
3. A *rooted Yang–Lu replacement*. The Yang–Lu inequality $R \geq \mathrm{diam}\,/2 + \sqrt{2} - 1$ is used in a rooted form, which shows that a shallow branch hanging from a bridge can be replaced by a path without increasing R or decreasing the radius.

Together these tools give the following picture of a minimal counterexample. It consists of a bridgeless core K in which every edge is supportive, together with pendant paths. Every core vertex has degree at least three, at most two core vertices have core-degree two, and at most one pendant path has length greater than one. Consequently $|V(K)| \leq 2\mu$, where μ is the cyclomatic number. For bounded μ the conjecture therefore reduces to a finite computation, which we carry out for $\mu \leq 8$. This covers 29,560,584 cores and 50,642,759 decorated configurations, with no counterexample.

In general we show the following. Every minimal counterexample contains a core vertex without pendant paths, adjacent to a core vertex with a pendant path, that is either a cut vertex of the core or a vertex of degree at least 7 whose deletion increases R. We also formulate a *Peeling Conjecture*: every non-tree is reducible either by a non-supportive edge or by removing all leaves. It implies the Caporossi–Hansen conjecture, and we verify it for all graphs on at most ten vertices.

Contents

1 Introduction **2**

2 Identities and elementary tools **4**

3 Trees and the rooted Yang–Lu inequality **6**

4 Minimal counterexamples **7**

5 Bounded cyclomatic number: proof of Theorem 1.4 **9**

6 The general case **11**

7 The Peeling Conjecture and computational evidence **12**

8 Concluding remarks **13**

A Code **14**
A.1 `bmu.c`: the finite verification for Theorem 1.4 14
A.2 `run_cyclomatic.sh`: driver 15
A.3 `xcheck.py`: independent brute-force cross-check 16
A.4 `peel10.c`: Peeling Conjecture for $n \le 10$ 17
A.5 `idcheck.py`, `brcheck.py`, `pathb.py`: sanity checks 17

1 Introduction

All graphs are finite and simple. For a connected graph G we write d_v for the degree of v, $\mathrm{ecc}(v) = \max_w d(v,w)$, $\mathrm{rad}(G) = \min_v \mathrm{ecc}(v)$, $\mathrm{diam}(G) = \max_v \mathrm{ecc}(v)$ and $\mu(G) = |E(G)| - |V(G)| + 1$ (the cyclomatic number). The *Randić index* [11] is

$$R(G) = \sum_{uv \in E(G)} \frac{1}{\sqrt{d_u d_v}}.$$

Write P_n and C_n for the path and the cycle on n vertices. Recall that $R(P_n) = \frac{n-3}{2} + \sqrt{2}$ for $n \ge 3$, that $R(C_n) = n/2$, and that $\mathrm{rad}(P_{2k}) = \mathrm{rad}(C_{2k}) = k$. In particular $R(P_{2k}) = k - (\frac{3}{2} - \sqrt{2}) < \mathrm{rad}(P_{2k})$ for $k \ge 2$, while $R(P_2) = \mathrm{rad}(P_2) = 1$ and $R(C_{2k}) = \mathrm{rad}(C_{2k})$.

Conjecture 1.1 (Fajtlowicz 1988 [8])**.** *$R(G) \ge \mathrm{rad}(G) - 1$ for every connected graph G.*

Conjecture 1.2 (Caporossi–Hansen 2000 [2])**.** *$R(G) \ge \mathrm{rad}(G)$ for every connected graph G that is not a path on an even number of vertices.*

Previous work. Caporossi and Hansen proved Conjecture 1.2 for trees [2]. Liu and Gutman [9] and You and Liu [13] treated unicyclic, bicyclic and tricyclic graphs, chemical graphs of cyclomatic number at most five, biregular graphs and graphs of order at most ten [7, 4]. We could not consult these two papers, and secondary sources differ on some details, so nothing below depends on them; Theorem 1.4 below recovers the cases of small cyclomatic number independently. Cygan, Pilipczuk and Škrekovski [3] proved Conjecture 1.2 for graphs of maximum degree at most four. Doig [5, 6] proved it for cacti. Deng, Tang and Zhang [4] proved $R(G) \ge \mathrm{rad}(G) - \frac{31}{105}(\mu(G) - 1)$ for $\mu(G) \ge 1$, using the harmonic index. For the diameter, Dvořák, Lidický and Škrekovski [7] proved $R(G) \ge \mathrm{diam}(G)/2$, and Yang and Lu [12] proved the sharp bound

$$R(G) \;\ge\; \frac{\mathrm{diam}(G)}{2} + \sqrt{2} - 1, \tag{1}$$

which settles a conjecture of Aouchiche, Hansen and Zheng. Since $\mathrm{diam} \ge \mathrm{rad}$, (1) gives only about $\mathrm{rad}\,/2$ in general. Conjecture 1.2 remains open, and so does the weaker Conjecture 1.1.

Results. We use three tools, developed in Sections 2–3.

(A) An exact edge-deletion identity (Lemma 2.1):

$$R(G-uv)-R(G)=\tfrac{1}{\sqrt{d_u d_v}}\big(\psi(d_u)A_u(v)+\psi(d_v)A_v(u)-1\big),$$

where $\psi(d)=\sqrt{d(d-1)}-(d-1)\in[0,\tfrac{1}{2})$ and $A_u(v)$ is an average of degree ratios. We call an edge *supportive* if its deletion increases R. A bridgeless graph always has a non-supportive edge (Lemma 2.3).

(B) A radius-stable peeling lemma (Lemma 2.6). If every vertex of S has a neighbour outside S and $G-S$ is connected, then $\mathrm{rad}(G-S)\geq\mathrm{rad}(G)-1$.

(C) A rooted form of the Yang–Lu inequality (Lemma 3.3). It yields a bridge replacement lemma (Lemma 3.4): a shallow branch behind a bridge can be replaced by a path without increasing R and without decreasing the radius.

We use these tools to analyse minimal counterexamples inside any class of graphs closed under connected subgraphs and under bridge replacement. Section 4 proves the following structure theorem.

Theorem 1.3 (Structure of minimal counterexamples)**.** *Let $\mathcal{C}$ be the class of all connected graphs, or the class of connected graphs of cyclomatic number at most c. Let G be a counterexample to Conjecture 1.2 in $\mathcal{C}$ with $|V(G)|$ minimum and, subject to this, $|E(G)|$ minimum. Put $r=\mathrm{rad}(G)$. Then:*

(i) *G consists of a bridgeless* core *K with $|V(K)|\geq 3$, together with pendant paths attached at core vertices. Every edge of K is supportive.*

(ii) *Every pendant-path end ℓ is the unique vertex at distance r from some vertex $c_\ell\in V(K)$ of eccentricity r. Pendant paths at the same core vertex have different lengths; in particular every core vertex carries at most one leaf.*

(iii) *At most one pendant path has length ≥ 2. If L is the set of leaves of G, then $\mathrm{rad}(G-L)=r-1$ and $R(G)-R(G-L)<1$.*

(iv) *Every core vertex has degree at least three in G. Every core vertex of core-degree two carries a pendant path. At most two core vertices have core-degree two.*

(v) *$|V(K)|\leq 2\mu(G)$. If G has a pendant path of length $s\geq 2$ at w, then $s\leq \mathrm{ecc}_K(w)$.*

Item (v) bounds the core in terms of the cyclomatic number, so for each fixed c the conjecture for $\mu\leq c$ becomes a finite problem. We solve it by computer for $c=8$ (Section 5).

Theorem 1.4. *Every connected graph G with $\mu(G)\leq 8$ that is not a path on an even number of vertices satisfies $R(G)\geq\mathrm{rad}(G)$.*

For general graphs we isolate exactly what remains (Section 6).

Theorem 1.5 (What a minimal counterexample must contain)**.** *Let G be a minimal counterexample to Conjecture 1.2 among all connected graphs, with core K as in Theorem 1.3. Then some core vertex b carries no pendant path, is adjacent to a core vertex carrying a pendant path, and satisfies one of the following:*

(a) *b is a cut vertex of K; or*

(b) *$d_G(b)\geq 7$ and $R(G-b)>R(G)$.*

Finally, we formulate the following conjecture.

Conjecture 1.6 (Peeling Conjecture)**.** *Let G be a connected graph that is not a tree and has at least three vertices, and let L be its set of leaves. Then either some edge e lying on a cycle satisfies $R(G-e) \le R(G)$, or*

$$R(G) - R(G-L) \;\ge\; \mathrm{rad}(G) - \mathrm{rad}(G-L).$$

Conjecture 1.6 implies Conjecture 1.2 (Proposition 7.1). We verified it for all connected graphs on at most 10 vertices. Extensive randomized searches on up to 60 vertices found no counterexample (Section 7).

What is and is not claimed. Theorems 1.3, 1.4 and 1.5 are proved here. Theorem 1.4 uses a computer enumeration whose code is reproduced in the appendix. The full Caporossi–Hansen conjecture, the Fajtlowicz conjecture and the Peeling Conjecture remain open.

2 Identities and elementary tools

Throughout, $f_v = d_v^{-1/2}$, so that $R(G) = \sum_{uv\in E} f_u f_v$. For $d \ge 1$ put

$$\psi(d) = \sqrt{d(d-1)} - (d-1), \qquad \varphi(d) = \frac{1}{\sqrt{d-1}} - \frac{1}{\sqrt{d}} \quad (d \ge 2).$$

Then $\psi(1) = 0$, ψ is increasing, and $0 \le \psi(d) < \tfrac{1}{2}$. Also $(d-1)\varphi(d) = \psi(d)/\sqrt{d}$, that is, $\varphi(d) = f\psi(d)/(d-1)$ when $f = d^{-1/2}$. Numerically,

$$\psi(2) = \sqrt{2}-1, \quad \psi(3) = \sqrt{6}-2 \approx 0.44949, \quad \psi(4) = 2\sqrt{3}-3 \approx 0.46410,$$

$$\psi(5) = 2\sqrt{5}-4 \approx 0.47214,$$

$$\varphi(2) = 1 - \tfrac{1}{\sqrt{2}} \approx 0.29289, \qquad \varphi(3) \approx 0.12976, \qquad \varphi(4) \approx 0.07735.$$

For an edge uv with $d_u \ge 2$ put

$$A_u(v) = \frac{1}{d_u - 1} \sum_{x \in N(u)\setminus\{v\}} \sqrt{\frac{d_v}{d_x}} = \frac{1}{d_u - 1} \sum_{x \in N(u)\setminus\{v\}} \frac{f_x}{f_v},$$

and set $\psi(d_u)A_u(v) = 0$ when $d_u = 1$.

Lemma 2.1 (Edge deletion)**.** *For every edge uv of a graph G,*

$$R(G-uv) - R(G) = f_u f_v\big(\psi(d_u)A_u(v) + \psi(d_v)A_v(u) - 1\big).$$

Proof. Only u and v change degree. The edge uv disappears, and each other edge ux changes weight by $(\frac{1}{\sqrt{d_u-1}} - \frac{1}{\sqrt{d_u}})f_x = \varphi(d_u)f_x$; the same holds at v. If x is a common neighbour of u and v, the edges ux and vx are distinct and d_x does not change. Hence $R(G-uv) - R(G) = -f_u f_v + \varphi(d_u)\sum_{x\in N(u)\setminus v} f_x + \varphi(d_v)\sum_{y\in N(v)\setminus u} f_y$. Substituting $\varphi(d_u) = f_u\psi(d_u)/(d_u-1)$ gives the formula. □

Definition 2.2. An edge e of G is *supportive* if $R(G-e) > R(G)$.

Lemma 2.3 (Bridgeless graphs have non-supportive edges)**.** *Let G be a connected bridgeless graph with at least three vertices. Then some edge e satisfies $R(G-e) < R(G)$.*

Proof. The minimum degree δ is at least two. Let u have degree δ and let v be a neighbour of u of minimum degree. Every $x \in N(u)$ has $d_x \geq d_v$, so $A_u(v) \leq 1$. Every $y \in N(v)$ has $d_y \geq \delta = d_u$, so $A_v(u) \leq 1$. Hence $\psi(d_u)A_u(v) + \psi(d_v)A_v(u) < \frac{1}{2} + \frac{1}{2} = 1$, and Lemma 2.1 gives the claim. □

Lemma 2.4 (Vertex deletion)**.** *Let b be a vertex such that every neighbour of b has degree at least two. Then*

$$R(G) - R(G-b) = \sum_{a \in N(b)} \Big(f_b f_a - \varphi(d_a) \sum_{y \in N(a) \setminus b} f_y \Big) - \sum_{\substack{aa' \in E \\ a,a' \in N(b)}} \varphi(d_a)\varphi(d_{a'}).$$

In particular, if ℓ is a leaf with neighbour a and $d_a \geq 2$, then

$$R(G) - R(G-\ell) = f_a - \varphi(d_a) \sum_{y \in N(a) \setminus \ell} f_y \;\geq\; \sqrt{d_a} - \sqrt{d_a - 1} > 0.$$

Proof. In $G - b$ exactly the neighbours of b lose one degree. For $a \in N(b)$ write $f'_a = f_a + \varphi(d_a)$. An edge ay with $a \in N(b)$ and $y \notin N(b) \cup \{b\}$ gains $\varphi(d_a) f_y$. An edge aa' with $a, a' \in N(b)$ gains $f'_a f'_{a'} - f_a f_{a'} = \varphi(d_a) f_{a'} + \varphi(d_{a'}) f_a + \varphi(d_a)\varphi(d_{a'})$. Summing gives the formula. For a leaf, the inequality follows from $f_y \leq 1$ and $(d-1)\varphi(d) = \sqrt{d-1} - \frac{d-1}{\sqrt{d}}$. □

Lemma 2.5 (Order versus radius)**.** *For connected G, $|V(G)| \geq 2\,\mathrm{rad}(G)$, with equality if and only if $G \in \{P_{2r}, C_{2r}\}$.*

Proof. Let T be a spanning tree. Distances do not decrease in T, so $\mathrm{rad}(T) \geq \mathrm{rad}(G)$. Every tree satisfies $\mathrm{diam}(T) \geq 2\,\mathrm{rad}(T) - 1$, so T, and hence G, contains a path on $2\,\mathrm{rad}(T) \geq 2\,\mathrm{rad}(G)$ vertices. Now suppose $|V(G)| = 2\,\mathrm{rad}(G)$. Then every spanning tree of G contains a path on all vertices, so every spanning tree is a path. If some vertex had degree at least three, a breadth-first search tree from it would not be a path. Hence G has maximum degree at most two, so G is a path or a cycle, and the radius condition forces P_{2r} or C_{2r}. □

Lemma 2.6 (Radius-stable peeling)**.** *Let G be connected, and let $S \subsetneq V(G)$ be such that $G - S$ is connected and every vertex of S has a neighbour outside S. Then* $\mathrm{rad}(G - S) \geq \mathrm{rad}(G) - 1$.

Proof. Let $x \in V(G) \setminus S$ and let y satisfy $d_G(x, y) \geq \mathrm{rad}(G)$. If $y \notin S$, then $d_{G-S}(x, y) \geq d_G(x, y) \geq \mathrm{rad}(G)$. Otherwise choose a neighbour $y' \notin S$ of y. Then $d_{G-S}(x, y') \geq d_G(x, y') \geq d_G(x, y) - 1 \geq \mathrm{rad}(G) - 1$. □

Lemma 2.7 (Peeling a set of leaves)**.** *Let L be a set of leaves of G with pairwise distinct neighbours. Let U be the set of these neighbours, and assume $d_u \geq 2$ for all $u \in U$. Write ℓ_u for the leaf of L at u, put $S_u = \sum_{y \in N(u) \setminus \ell_u} f_y$ and $t_u = f_u - \varphi(d_u) S_u$. Then*

$$\mathrm{drop}(L) := R(G) - R(G - L) = \sum_{u \in U} \Big(t_u - \tfrac{1}{2}\varphi(d_u) \sum_{v \in U \cap N(u)} \varphi(d_v) \Big).$$

Moreover, $\mathrm{drop}(L) > 0$*, and* drop *is strictly increasing under adding leaves. More precisely, if G^+ is obtained from G by attaching a new leaf at a vertex of $G - L$ that does not already carry a leaf of L, and $L^+ = L \cup \{\text{new leaf}\}$, then $G^+ - L^+ = G - L$ and $R(G^+) > R(G)$, so that* $\mathrm{drop}_{G^+}(L^+) > \mathrm{drop}_G(L)$.

Proof. The formula is the computation in the proof of Lemma 2.4, applied to all the removed leaves at once. Each leaf edge $u\ell_u$ contributes f_u, and the double sum collects the second-order terms of edges between two supports. Positivity follows by removing the leaves one at a time, using Lemma 2.4. For monotonicity, $G^+ - L^+ = G - L$ holds by construction, and $R(G^+) > R(G)$ is Lemma 2.4 applied to the new leaf in G^+. □

3 Trees and the rooted Yang–Lu inequality

Lemma 3.1 (Trees)**.** *If T is a tree with at least two vertices that is not a path on an even number of vertices, then $R(T) \geq \operatorname{rad}(T) + 0.27$.*

Proof. Let P be a diametral path of T, with $D+1$ vertices. As long as $T \neq P$, the tree has a leaf off P (take a vertex off P farthest from P). Deleting it strictly decreases R (Lemma 2.4) and keeps P diametral. Hence $R(T) \geq R(P_{D+1})$, and if $T \neq P$ then $R(T) \geq R(T_1)$, where T_1 is P with one leaf attached at an internal vertex. Also $D \geq 2r-1$ with $r = \operatorname{rad}(T)$. If $D \geq 2r$ then $R(T) \geq R(P_{2r+1}) = r + \sqrt{2} - 1$. If $D = 2r-1$ and $T \neq P_{2r}$, attaching a leaf to an internal vertex of P_{2r} increases R by at least $\frac{2}{\sqrt{3}} + \frac{1}{\sqrt{6}} - \frac{1}{\sqrt{2}} - \frac{1}{2} > 0.3558$ (the minimum is attained at the second vertex of the path). Hence $R(T) \geq R(P_{2r}) + 0.3558 > r + 0.27$. □

Theorem 3.2 (Yang–Lu [12])**.** *For every connected graph H with at least two vertices, $R(H) \geq \frac{1}{2}\operatorname{diam}(H) + \sqrt{2} - 1$.*

For a connected graph H and $y \in V(H)$, let $R^{+y}(H)$ denote the Randić sum over $E(H)$ computed with the degree of y increased by one, that is, as if y had one extra neighbour outside H.

Lemma 3.3 (Rooted Yang–Lu)**.** *Let H be connected with at least two vertices, let $y \in V(H)$, and put $e = \operatorname{ecc}_H(y)$. Then*

$$R^{+y}(H) \;\geq\; \frac{e}{2} + \frac{1 - (d_H(y)+1)^{-1/2}}{\sqrt{2}}.$$

Proof. Attach to H a new path $y\,p\,q$ and call the result H'. Then $\operatorname{diam}(H') \geq e+2$, and $R(H') = R^{+y}(H) + \frac{(d_H(y)+1)^{-1/2}}{\sqrt{2}} + \frac{1}{\sqrt{2}}$. Theorem 3.2 gives $R(H') \geq \frac{e+2}{2} + \sqrt{2} - 1$, and rearranging proves the lemma. □

Lemma 3.4 (Bridge replacement)**.** *Let xy be a bridge of a connected graph G, and let $G_x \ni x$ and $G_y \ni y$ be the two components of $G - xy$. Put $e_x = \operatorname{ecc}_{G_x}(x)$ and $e_y = \operatorname{ecc}_{G_y}(y)$, and suppose $e_x \geq e_y$ and $|V(G_x)| \geq 2$. Let G' be obtained from G_x by attaching at x a new path $x\,p_1 \cdots p_{e_y+1}$. Then*

$$R(G') \leq R(G), \qquad \operatorname{rad}(G') \geq \operatorname{rad}(G), \qquad \mu(G') \leq \mu(G), \qquad |V(G')| \leq |V(G)|,$$

and equality $|V(G')| = |V(G)|$ holds only if G_y is a path having y as an end vertex, in which case $G' \cong G$.

Proof. The vertex x has the same degree in G and G', so all edges of G_x keep their weights. If $e_y = 0$, then $G' \cong G$. Suppose $e_y \geq 1$. The bridge side of G contributes $f_x f_y + R^{+y}(G_y)$, while the path side of G' contributes $\frac{f_x}{\sqrt{2}} + \frac{e_y - 1}{2} + \frac{1}{\sqrt{2}}$. Here $f_x \leq \frac{1}{\sqrt{2}}$ because $d_G(x) \geq 2$, and $f_y = (d_{G_y}(y)+1)^{-1/2} \leq \frac{1}{\sqrt{2}}$. Hence

$$R(G) - R(G') \;\geq\; R^{+y}(G_y) - \tfrac{1}{\sqrt{2}}\bigl(\tfrac{1}{\sqrt{2}} - f_y\bigr) - \tfrac{e_y-1}{2} - \tfrac{1}{\sqrt{2}} = R^{+y}(G_y) - \tfrac{e_y}{2} - \tfrac{1-f_y}{\sqrt{2}} \;\geq 0$$

by Lemma 3.3.

For the radius, take $z \in V(G_x)$. Every path from z into G_y passes through x, so $\operatorname{ecc}_{G'}(z) = \max(\operatorname{ecc}_{G_x}(z), d(z,x) + e_y + 1) = \operatorname{ecc}_G(z)$. For a new vertex p_h we have $\operatorname{ecc}_{G'}(p_h) \geq h + e_x \geq 1 + e_x \geq \max(e_x, e_y + 1) = \operatorname{ecc}_G(x) \geq \operatorname{rad}(G)$. Hence $\operatorname{rad}(G') \geq \operatorname{rad}(G)$.

The graph G' is G_x plus a tree, so $\mu(G') = \mu(G_x) \leq \mu(G)$. Finally $|V(G_y)| \geq e_y + 1$, with equality exactly when G_y is a path starting at y. □

4 Minimal counterexamples

Fix $\mathcal{C}$ to be either the class of all connected graphs or the class $\mathcal{C}_c$ of connected graphs with $\mu \le c$. Both classes are closed under taking connected subgraphs and under the replacement of Lemma 3.4.

A *counterexample* is a graph $G \in \mathcal{C}$ that is not an even path and satisfies $R(G) < \mathrm{rad}(G)$. A counterexample is *minimal* if no counterexample in $\mathcal{C}$ has fewer vertices, or the same number of vertices and fewer edges. Throughout this section G is a minimal counterexample, $r = \mathrm{rad}(G)$ and $n = |V(G)|$. Every graph $H \in \mathcal{C}$ that is smaller in this order satisfies $R(H) \ge \mathrm{rad}(H)$ unless H is an even path, in which case $R(H) \ge \mathrm{rad}(H) - (\frac{3}{2} - \sqrt{2})$.

Proposition 4.1 (Edges)**.** *G is not a tree, and every edge of G lying on a cycle is supportive.*

Proof. Lemma 3.1 excludes trees. Let e lie on a cycle and suppose $R(G-e) \le R(G)$. The graph $G-e$ is connected, lies in $\mathcal{C}$, has fewer edges, and satisfies $\mathrm{rad}(G-e) \ge r$. If $G-e$ is not an even path, then $R(G) \ge R(G-e) \ge \mathrm{rad}(G-e) \ge r$, a contradiction. If $G-e = P_{2k}$, then $n = 2k$ and $r \le k$. If $r = k$, Lemma 2.5 forces $G = C_{2k}$, which has $R = r$. Otherwise $R(G) \ge R(P_{2k}) = k - \frac{3}{2} + \sqrt{2} > k - 1 \ge r$. Either way we have a contradiction. □

Proposition 4.2 (Bridges)**.** *For every bridge xy, with notation as in Lemma 3.4, we have $e_x \ne e_y$. If $e_x > e_y$, then G_y is a path having y as an end vertex.*

Proof. Assume $e_x \ge e_y$. If $G_x = \{x\}$, then $e_y = 0$ and $G = K_2$, a tree, which is excluded by Proposition 4.1. So $|V(G_x)| \ge 2$. Suppose that G_y is not a path ending at y. Lemma 3.4 gives $G' \in \mathcal{C}$ with fewer vertices, $R(G') \le R(G) < r \le \mathrm{rad}(G')$. Since G is minimal, G' is an even path P_{2k}. Then G_x is a path with end x, so G is G_y with a pendant path $Q = y\,x\cdots$ of length $L = e_x + 1$ attached at y, and $2k = e_x + e_y + 2$. Since $R(G) \ge R(P_{2k}) > k - 1$ and $r \le \mathrm{rad}(G') = k$, we may assume $r = k$.

Suppose first that $L \ge 3$. Delete the last two vertices of Q and call the result G''; then $G'' \in \mathcal{C}$. For every vertex z of G'', at distance t from y (inside G_y or on the shortened tail), $\mathrm{ecc}_{G''}(z) \ge \max(t + L - 2,\, e_y - t)$ if $z \in V(G_y)$, and $\mathrm{ecc}_{G''}(z) \ge \max(L - 2 - t,\, t + e_y)$ if z is on the tail. Hence $\mathrm{rad}(G'') \ge \lceil \frac{L-2+e_y}{2} \rceil = k - 1$. Write the tail as $y = t_0, t_1, \ldots, t_L$. Removing t_L and t_{L-1} deletes edges of weights $\frac{1}{\sqrt{2}}$ and $\frac{1}{2}$, and turns t_{L-2} into a leaf. This increases the weight of the edge $t_{L-3}t_{L-2}$ by $f_{t_{L-3}}(1 - \frac{1}{\sqrt{2}})$. Since $d(t_{L-3}) \ge 2$, the total decrease of R is $\frac{1}{\sqrt{2}} + \frac{1}{2} - f_{t_{L-3}}(1 - \frac{1}{\sqrt{2}}) \ge 1$. The graph G'' is not a path, because G_y is not a path ending at y. Thus $R(G) \ge R(G'') + 1 \ge k$, a contradiction.

If $L \le 2$, then $e_y \le e_x \le 1$ and $k \le 2$. Now $R(G) \ge \sqrt{n-1}$ by the Bollobás–Erdős bound [1], and $n \ge 2k + 1$, so $R(G) \ge k$, again a contradiction.

So G_y is a path ending at y. If $e_x = e_y$, the symmetric argument shows that G_x is a path ending at x, so G is a path, contradicting Proposition 4.1. □

Proposition 4.3 (Core and pendant paths)**.** *Let K be the 2-core of G, that is, the subgraph left after repeatedly deleting leaves. Then $|V(K)| \ge 3$, K is bridgeless, and $G - E(K)$ consists of pendant paths. Each one is attached at one of its end vertices to a single vertex $w \in V(K)$, and its length is at most the eccentricity of w in the graph obtained from G by deleting the path (except w). No vertex outside K has eccentricity r.*

Proof. K is nonempty because G is not a tree. Each tree of $G - E(K)$ hanging at a core vertex w is attached through bridges. Consider the bridge wy joining w to such a tree. Its side containing w contains the cycles of K, so it is not a path. By Proposition 4.2 we therefore have $e_w > e_y$, and the side of y is a path starting at y. So the branch is a pendant path of length $e_y + 1 \le e_w$. Suppose xy is a bridge with $x, y \in V(K)$. The vertices of K on the side of x induce a graph in which every vertex except possibly x has degree at least two, so that side contains a cycle;

the same holds on the side of y. Then neither side is a path, which Proposition 4.2 forbids. Hence K is bridgeless. Finally, let z lie on a pendant path of length s at distance $h \geq 1$ from its attachment vertex w, and let e_w be the eccentricity of w in the rest of G. Since $s \leq e_w$, we get $\mathrm{ecc}(z) \geq h + e_w > e_w = \mathrm{ecc}(w) \geq r$. □

We call a core vertex *decorated* if it carries a pendant path, and *bare* otherwise. A pendant path of length at least two is *long*.

Proposition 4.4 (Leaves are radius-critical)**.** *Let ℓ be the end of a pendant path. Then there is a vertex $c_\ell \in V(K)$ with $\mathrm{ecc}(c_\ell) = r$ such that ℓ is the only vertex at distance r from c_ℓ. Two pendant paths at the same core vertex have different lengths. In particular, every core vertex carries at most one leaf.*

Proof. The graph $G - \ell$ lies in $\mathcal{C}$, is smaller, and is not a path (it contains K). By Lemma 2.4, $R(G-\ell) < R(G) < r$. If $\mathrm{rad}(G-\ell) \geq r$, then $G - \ell$ would be a smaller counterexample; hence $\mathrm{rad}(G-\ell) \leq r-1$. Let c have $\mathrm{ecc}_{G-\ell}(c) \leq r-1$. Deleting a leaf does not change distances between the remaining vertices, so in G every vertex other than ℓ lies within $r-1$ of c. Since $\mathrm{ecc}_G(c) \geq r$, we get $d(c,\ell) = r$, and $c \in V(K)$ by Proposition 4.3.

If two pendant paths at w had the same length, their ends would be at equal distance from every core vertex, so neither could be the unique farthest vertex of a central core vertex. □

Proposition 4.5 (Peeling)**.** *Let L be the set of leaves of G. Then* $\mathrm{rad}(G-L) = r-1$ *and* $\mathrm{drop}(L) < 1$. *Moreover, G has at most one long pendant path.*

Proof. First, $L \neq \emptyset$: otherwise $G = K$ would be bridgeless (Proposition 4.3), and Lemma 2.3 would give a non-supportive edge on a cycle, contradicting Proposition 4.1. The supports of the leaves are pairwise distinct (Proposition 4.4) and none of them is a leaf, so Lemma 2.6 gives $\mathrm{rad}(G-L) \geq r-1$. Lemma 2.7 gives $\mathrm{drop}(L) > 0$. The graph $G - L \in \mathcal{C}$ is smaller and contains K, so it is not a path. If $\mathrm{rad}(G-L) \geq r$, it would be a smaller counterexample. Hence $\mathrm{rad}(G-L) = r-1$. If $\mathrm{drop}(L) \geq 1$, then $R(G) \geq R(G-L)+1 \geq r$, a contradiction.

Suppose two long pendant paths exist, with ends q_1, q_2 and supports p_1, p_2 of degree two. Put $S = \{q_1, q_2\}$. Lemma 2.6 applies, so $\mathrm{rad}(G-S) \geq r-1$, and $G - S$ is smaller and not a path. The supports p_1, p_2 are not adjacent, and Lemma 2.7 gives

$$\mathrm{drop}(S) = \sum_{i=1,2} \left(\tfrac{1}{\sqrt{2}} - \varphi(2) f_{z_i}\right) \;\geq\; 2\left(\tfrac{1}{\sqrt{2}} - \varphi(2)\tfrac{1}{\sqrt{2}}\right) = 1,$$

where z_i is the other neighbour of p_i, which has degree at least two. Hence $R(G) \geq R(G-S)+1 \geq r$, a contradiction. □

Proposition 4.6 (Core degrees)**.** *Every bare core vertex has core-degree at least three. Consequently every core vertex has degree at least three in G, and at most two core vertices have core-degree two.*

Proof. Let x be bare with core-degree two, and let v_1, v_2 be its neighbours with $d_{v_1} \leq d_{v_2}$. Then $A_x(v_1) = \sqrt{d_{v_1}/d_{v_2}} \leq 1$. By Proposition 4.1 the edge xv_1 is supportive, so Lemma 2.1 gives $\psi(d_{v_1}) A_{v_1}(x) > 1 - \psi(2) = 2 - \sqrt{2} \approx 0.58579$.

Now $A_{v_1}(x) = \frac{1}{d_{v_1}-1} \sum_{y \in N(v_1)\setminus x} \sqrt{2/d_y}$. Among these y there is at most one leaf (giving $\sqrt{2}$, by Proposition 4.4). All other y have degree at least two (giving at most 1). Hence $A_{v_1}(x) \leq \alpha(d) := \frac{\sqrt{2}+d-2}{d-1}$ with $d = d_{v_1}$. If $d = 2$, both neighbours of v_1 are core vertices (core-degree is at least two), so $A_{v_1}(x) \leq 1$ and $\psi(2) \cdot 1 < 0.586$. For $d = 3$, $\psi(3)\alpha(3) \approx 0.54258$. For $d = 4$, $\psi(4)\alpha(4) \approx 0.52818$. For $d \geq 5$, $\psi(d)\alpha(d) < \frac{1}{2}\alpha(5) < 0.552$. All of these are below $2 - \sqrt{2}$, a contradiction.

So every core vertex has degree at least three: a bare one through core-degree ≥ 3, a decorated one through core-degree ≥ 2 plus a pendant neighbour.

For the last claim, write the formula of Lemma 2.7 for $\mathrm{drop}(L)$ as a sum of brackets, one per support. Since non-leaf neighbours have degree ≥ 2, $t_u \geq f_u(1-\psi(d_u)/\sqrt{2}) \geq 0.646 f_u$. The other supports adjacent to u have degree ≥ 3, except possibly the support p of a long path, which has degree two and $\varphi(2) \approx 0.293$. Using $\varphi(d) \leq \varphi(3)$ for $d \geq 3$, every bracket is at least $f_u(1-\psi(d_u)/\sqrt{2}) - \frac{1}{2} f_u \psi(d_u)(\varphi(3) + \varphi(2)/(d_u - 1)) > 0.6 f_u > 0$. The bracket of p is at least $\frac{1}{2} - \frac{1}{2}\varphi(2)\varphi(4) > 0.488$, because $t_p \geq \frac{1}{\sqrt{2}}(1-\varphi(2)) = \frac{1}{2}$ and the only possible support adjacent to p is w, of degree at least four.

A support u of core-degree two and degree three has core neighbours of degree ≥ 3. Hence $t_u \geq \frac{1}{\sqrt{3}} - \varphi(3)\frac{2}{\sqrt{3}} > 0.4275$ and its bracket is at least $0.4275 - \frac{1}{2}\varphi(3) \cdot 2\varphi(3) > 0.410$.

Suppose that three core vertices have core-degree two. All of them are decorated. Each of them other than w carries exactly one leaf and has degree three, so its bracket is at least 0.410. If w is one of them, then the long path exists and the bracket of p is at least 0.488. Since all brackets are positive, $\mathrm{drop}(L) \geq \min(3 \cdot 0.410,\ 2 \cdot 0.410 + 0.488) > 1$, contradicting Proposition 4.5. □

Corollary 4.7. *$|V(K)| \leq 2\mu(G)$. If a long pendant path of length s is attached at w, then $2 \leq s \leq \mathrm{ecc}_K(w)$.*

Proof. Pendant paths create no cycles, so $\mu(K) = \mu(G)$ and $\sum_{v \in K}(d_K(v) - 2) = 2\mu(G) - 2$. At least $|V(K)| - 2$ core vertices have core-degree at least three, so $|V(K)| - 2 \leq 2\mu(G) - 2$.

For the second claim, let $w = t_0, t_1, \ldots, t_s = q$ be the long path and let e be the eccentricity of w in $G - \{t_1, \ldots, t_s\}$. Distances between core vertices are the same in K and in G, and by Proposition 4.5 every other pendant path is a single leaf at a core vertex. Hence $e \leq \mathrm{ecc}_K(w) + 1$. Proposition 4.3 gives $s \leq e$, so $\mathrm{ecc}(w) = e$. By Proposition 4.4 there is a core vertex c with $\mathrm{ecc}(c) = r$ such that q is the only vertex at distance r from c. Then $r = d(c, w) + s$ and $r \leq \mathrm{ecc}(w) = e$. If $c = w$, then $r = s \leq e \leq r$. So $e = s = r$, and a vertex outside the long path at distance e from w contradicts the uniqueness of q. Hence $d(c, w) \geq 1$, and $s = r - d(c, w) \leq e - 1 \leq \mathrm{ecc}_K(w)$. □

Proof of Theorem 1.3. Combine Propositions 4.1–4.6 and Corollary 4.7. □

5 Bounded cyclomatic number: proof of Theorem 1.4

Let G be a minimal counterexample in $\mathcal{C}_8$. By Theorem 1.3, G is obtained from a core K as follows.

- K is a connected bridgeless graph with minimum degree at least two and at most two vertices of degree two. It satisfies $3 \leq |V(K)| \leq 2\mu$ and $|E(K)| = |V(K)| + \mu - 1$ for some $1 \leq \mu \leq 8$.
- A set $U \subseteq V(K)$ of core vertices each receives one leaf.
- Optionally, one core vertex w receives a pendant path of length s with $2 \leq s \leq \mathrm{ecc}_K(w)$.
- Every vertex of degree two in K lies in $U \cup \{w\}$.

Moreover, G satisfies $\mathrm{drop}(L) < 1$, every edge of K is supportive, and $R(G) < \mathrm{rad}(G)$.

Algorithm. For each admissible pair $(|V(K)|, \mu)$, nauty's `geng` [10] enumerates all connected graphs with minimum degree at least two and the prescribed numbers of vertices and edges. The program `bmu` (Appendix A) keeps the bridgeless ones with at most three vertices of degree two. These conditions, and the range $|V(K)| \le 2\mu + 1$ used by the driver, are weaker than those of Theorem 1.3; this only enlarges the search (the runs were made before we noticed the sharper count in Proposition 4.6). The driver skips pairs with $|E(K)| < \lceil (3|V(K)| - 3)/2 \rceil$, which admit no core with at most three vertices of degree two. For each choice of w and s (or none), it enumerates U by depth-first search. Lemma 2.7 says that $\mathrm{drop}(L)$ is strictly increasing as leaves are added, so a branch can be pruned as soon as $\mathrm{drop}(L) \ge 1$. Every surviving configuration is tested for supportiveness of all core edges. Every configuration that passes is tested for $R(G) \ge \mathrm{rad}(G)$.

All comparisons are made conservatively in floating point:

- a configuration is discarded as having a non-supportive edge only if $R(G - e) - R(G) < -10^{-9}$;
- pruning happens only when $\mathrm{drop}(L) \ge 1 + 10^{-9}$;
- a configuration is reported as a violation whenever $R(G) < \mathrm{rad}(G) + 10^{-9}$.

Results. Table 1 lists the counts. Over all $\mu \le 8$ the program examined 29,560,584 cores and 50,642,759 decorated configurations with $\mathrm{drop}(L) < 1$. Exactly one configuration has all core edges supportive. Its core is $K_{3,4}$, with one leaf at each of the three vertices of the smaller part. This graph has 10 vertices, radius 2, $R = 4.4400$ and $\mathrm{drop}(L) = 0.9759$, and each core edge e has $R(G - e) - R(G) = 0.0021$. No violation occurred. Hence no minimal counterexample exists in $\mathcal{C}_8$, which proves Theorem 1.4.

Cross-check. The script `xcheck.py` (Appendix A) is an independent brute-force implementation of the same search. It uses no pruning and no edge-deletion formula; it enumerates every leaf set U and recomputes R, $\mathrm{drop}(L)$, $R(G - e)$ and the radius from scratch. For $\mu \le 4$ it reproduces the counts of Table 1 exactly. As a consistency check of Proposition 4.6, we also reran `bmu` on the 974,824 cores with $\mu \le 7$ and exactly three vertices of degree two: none of them admits a configuration with $\mathrm{drop}(L) < 1$.

μ	cores	configurations with drop < 1	all core edges supportive	violations
1	1	0	0	0
2	3	3	0	0
3	21	45	0	0
4	194	523	0	0
5	2 675	6 182	0	0
6	48 739	97 983	1	0
7	1 089 292	1 984 374	0	0
8	28 419 659	48 553 649	0	0
total	29 560 584	50 642 759	1	0

Table 1: The finite verification behind Theorem 1.4.

Remark 5.1. On four cores the run for $\mu = 7$ took a few minutes and the run for $\mu = 8$ about one hour. For $\mu = 8$ and $|V(K)| \ge 15$ the driver passed to `geng` the maximum-degree bound `-D`$(2\mu + 4 - |V(K)|)$, which every core accepted by the filter of `bmu` satisfies. The driver in the appendix uses this bound throughout; for $\mu \le 6$ we checked that the counts are identical with

and without it. The row $\mu = 8$ includes the 599,021 cores with $|V(K)| = 17 = 2\mu + 1$, which Proposition 4.6 shows to be unnecessary; none of them admits a configuration with $\mathrm{drop}(L) < 1$. The bottleneck for larger μ is the number of cores with $|V(K)|$ close to 2μ.

6 The general case

In this section $\mathcal{C}$ is the class of all connected graphs and G is a minimal counterexample, with core K as in Theorem 1.3.

Lemma 6.1 (Corona lemma)**.** *If every core vertex is decorated, then* $\mathrm{drop}(L) \geq 1$.

Proof. All core vertices except possibly w (the attachment vertex of the long path, if there is one) carry a leaf. For a core support $u \neq w$, all neighbours other than its leaf are core vertices of degree ≥ 3. Hence $t_u \geq f_u(1 - \psi(d_u)/\sqrt{3}) \geq 0.711 f_u$, and the second-order part of its bracket is at most $\frac{1}{2} f_u \psi(d_u)\varphi(3) \leq 0.033 f_u$, so its bracket is at least $0.678 f_u$. Since $d_u \leq |V(K)|$,

$$\sum_{u \in V(K) \setminus \{w\}} f_u \;\geq\; \frac{|V(K)| - 1}{\sqrt{|V(K)|}}.$$

Without a long path the sum runs over all of $V(K)$, and $\mathrm{drop}(L) \geq 0.678\sqrt{3} > 1.17$. With a long path, the support of its end contributes a bracket of at least 0.488, the bracket of w is non-negative, and $\mathrm{drop}(L) \geq 0.678 \cdot \frac{2}{\sqrt{3}} + 0.488 > 1.27$. □

Lemma 6.2 (Bare vertices whose deletion lowers R are radius-critical)**.** *Let b be a bare core vertex that is not a cut vertex of K, and suppose $R(G - b) \leq R(G)$. Then b is the only vertex at distance r from some vertex of eccentricity r. In particular, no neighbour of b is decorated.*

Proof. The graph $G - b$ is connected, because b carries no pendant path. Suppose first that $G - b$ is not an even path. Minimality then forces $\mathrm{rad}(G - b) \leq r - 1$, so some c has all vertices of $G - b$ within distance $r - 1$ in $G - b$, and hence within distance $r - 1$ in G. Since $\mathrm{ecc}_G(c) \geq r$ and b has a neighbour within distance $r - 1$ of c, we get $d(c, b) = r = \mathrm{ecc}_G(c)$, and b is the unique vertex at distance r from c. By Proposition 4.3, $c \in V(K)$.

Suppose instead that $G - b = P_{2k} = v_1 v_2 \cdots v_{2k}$. Then $n = 2k + 1$, so $r \leq k$, and $R(G) \geq R(G - b) = R(P_{2k}) > k - 1$. So we may assume $r = k$. Let v_p and v_q $(p < q)$ be the first and last neighbours of b on the path. The core of G is $G[\{b, v_p, \ldots, v_q\}]$, and $v_1 \cdots v_{p-1}$ and $v_{q+1} \cdots v_{2k}$ are its pendant paths. By Proposition 4.6, every core vertex of core-degree two is decorated. Hence $p \geq 2$, $q \leq 2k - 1$, and every v_j with $p < j < q$ is adjacent to b. In particular all of $v_p, \ldots, v_q$ have degree three, and $d_b = q - p + 1$. If $q - p \geq 3$, consider the edge $v_{p+1}v_{p+2}$. Each endpoint has degree three, and its two other neighbours have degree at least three, so $A_{v_{p+1}}(v_{p+2}) \leq 1$ and $A_{v_{p+2}}(v_{p+1}) \leq 1$. By Lemma 2.1 this edge is not supportive, contradicting Proposition 4.1. Since $d_b \geq 3$ (Proposition 4.6), we conclude that $q - p = 2$: the vertex b is adjacent to exactly three consecutive vertices of the path. The change $R(G) - R(P_{2k})$ depends only on the position of these three vertices relative to the ends of the path. So it suffices to check every position for $2 \leq k \leq 39$, which covers every local type. The script `pathb.py` finds $R(G) - k \geq 0.359$ in all cases, contradicting $R(G) < r = k$.

Finally, in the first case suppose that $u \in N(b)$ is decorated by a pendant path of length $s \geq 1$ with end ℓ. Then $\ell \neq b$, so $d(c, \ell) \leq r - 1$. Hence $d(c, u) \leq r - 1 - s$ and $d(c, b) \leq r - s \leq r - 1$, a contradiction. □

Lemma 6.3 (Hubs have degree at least seven)**.** *If b is a bare core vertex with $R(G - b) > R(G)$, then $d_b \geq 7$.*

Proof. Use Lemma 2.4. For $a \in N(b)$, every neighbour of a other than b is one of the following: a core vertex of degree ≥ 3, the unique leaf at a, or the first vertex of the unique long path. This gives

$$\varphi(d_a) \sum_{y \in N(a) \setminus b} f_y = f_a \frac{\psi(d_a)}{d_a - 1} \sum_y f_y \;\leq\; \kappa f_a,$$

where, writing $\beta(d) = 1$ for $d \geq 4$ and $\beta(3) = 0$,

$$\kappa = \max_{d \geq 3} \frac{\psi(d)}{d-1}\Big(1 + \frac{\beta(d)}{\sqrt{2}} + \frac{d - 2 - \beta(d)}{\sqrt{3}}\Big) = \frac{\psi(3)}{2}\Big(1 + \frac{1}{\sqrt{3}}\Big) < 0.3546.$$

(For $d = 3$ at most one neighbour of a other than b lies outside the core; for $d \geq 4$ at most two do, namely a leaf and the first vertex of the long path. The values for $d = 4, 5, 6$ are below 0.354, 0.338 and 0.329. For $d \geq 7$, the bound $\psi(d) < \frac{1}{2}$ shows that the expression is below $(1 + 0.957/(d-1))/(2\sqrt{3}) < 0.335$.) The second-order sum is at most $\frac{1}{2}\sum_a \varphi(d_a)(d_a - 1)\varphi(3) \leq \frac{1}{4}\varphi(3)\sum_a f_a < 0.0325 \sum_a f_a$. Hence

$$R(G) - R(G - b) \;\geq\; \sum_{a \in N(b)} f_a(f_b - 0.3871),$$

which is positive when $d_b \leq 6$, because then $f_b \geq 6^{-1/2} > 0.408$. □

Proof of Theorem 1.5. If no core vertex were decorated, then $G = K$ would be bridgeless. Lemma 2.3 would then give a non-supportive edge, contradicting Proposition 4.1. If every core vertex were decorated, Lemma 6.1 would contradict Proposition 4.5. Since K is connected, some bare core vertex b is adjacent to a decorated core vertex. By Lemma 6.2, either b is a cut vertex of K, or $R(G - b) > R(G)$, in which case $d_b \geq 7$ by Lemma 6.3. □

Remark 6.4. A cut vertex of the bridgeless core has core-degree at least four, since each component of $K - b$ sends at least two edges to b. So in both alternatives of Theorem 1.5, b has large degree. This is consistent with the theorem of Cygan, Pilipczuk and Škrekovski for maximum degree four. Excluding the two alternatives would prove Conjecture 1.2. For alternative (a), we believe that the following inequality is the key step: if Q is a connected graph, $z \in V(Q)$ with $\deg_Q(z) = j \geq 2$ and $Q - z$ connected, then

$$R(Q + \text{pendant } P_2 \text{ at } z) \;\geq\; R(S_{e,2,1^{j-1}}),$$

where $e = \mathrm{ecc}_Q(z)$ and $S_{e,2,1^{j-1}}$ is the spider with legs of lengths $e, 2, 1, \ldots, 1$ ($j - 1$ ones). The intended use is to replace the branch of smallest eccentricity at a core cut vertex by such a spider, which should reduce the order (or keep it and remove a cycle) without increasing R or decreasing the radius; we have not written out this reduction in full, so we do not claim that the inequality implies the exclusion of (a). We verified the inequality for all rooted graphs on at most 9 vertices, with minimum margin 0.0176. We could not prove it in general.

7 The Peeling Conjecture and computational evidence

Proposition 7.1. *Conjecture 1.6 implies Conjecture 1.2.*

Proof. Let G be a minimal counterexample among all connected graphs. By Proposition 4.1, G is not a tree and has no non-supportive edge on a cycle. The set L of leaves is nonempty: otherwise $G = K$ would be bridgeless (Proposition 4.3), and Lemma 2.3 would contradict Proposition 4.1. So $G - L$ is connected and smaller than G. Conjecture 1.6 then gives $R(G) - R(G - L) \geq \mathrm{rad}(G) - \mathrm{rad}(G - L)$. The graph $G - L$ contains a cycle of G, so it is not a path, and since it is smaller, $R(G - L) \geq \mathrm{rad}(G - L)$. Adding the two inequalities gives $R(G) \geq \mathrm{rad}(G)$, a contradiction. □

Exhaustive check for $n \le 10$. For every connected graph on at most 10 vertices (generated by `geng`), the program `peel10.c` tests whether every cycle edge is supportive, conservatively. The numbers of such non-tree graphs for $n = 6, \dots, 10$ are $1, 3, 15, 43, 161$, and all of them satisfy the peeling inequality of Conjecture 1.6. The smallest value of $\mathrm{drop}(L)$ among them is 0.9759, attained by $K_{3,4}$ with one leaf at each vertex of the smaller part (the graph found in Section 5). That graph does not lose radius when its leaves are removed, so it satisfies the inequality anyway. (For a graph without leaves the inequality is trivial; the program would flag any such graph with all cycle edges supportive, and found none. The program also adds a margin for the case that $G - L$ is an even path, which cannot occur because $G - L$ contains a cycle.)

Randomized searches. Simulated annealing over graphs with up to 60 vertices produced tens of thousands of graphs in which every cycle edge is supportive. The objectives included the peeling margin $\mathrm{drop}(L) - (\mathrm{rad}(G) - \mathrm{rad}(G - L))$, and some runs were seeded with high-degree hubs. No violation of Conjecture 1.6 was found. The smallest margins observed were about 0.17, in dense cores with three leaves and radius 3. Each of these graphs had a bare vertex adjacent to a support whose deletion lowers R while preserving the radius, so it is excluded from minimal counterexamples by Lemma 6.2. Searches imposing *all* the properties of Theorem 1.3 together with Lemma 6.2 produced only coronas of regular graphs. Those are excluded by Lemma 6.1.

Sanity checks. The script `idcheck.py` tests Lemma 2.1, Lemma 2.4 (the general formula and the leaf inequality), Lemma 2.6 and Lemma 2.7 (formula, positivity and monotonicity) on random graphs, in 50,691 instances. The script `brcheck.py` tests Lemmas 3.3 and 3.4 in 8,233 and 5,450 instances. No discrepancy occurred.

8 Concluding remarks

The method replaces a global inequality between a degree-based index and a metric parameter by three local mechanisms. Supportive edges control edge deletion, radius-stable peeling controls vertex deletion, and rooted Yang–Lu replacement controls bridges. What survives is a sharply constrained core. In bounded cyclomatic number, the core has at most 2μ vertices and the conjecture becomes a finite problem. In general, the obstruction is concentrated at high-degree bare vertices next to decorated vertices: cut vertices of the core, or hubs whose removal increases R.

Excluding both alternatives of Theorem 1.5 would complete a proof of the Caporossi–Hansen conjecture. We record two problems aimed at them.

1. Exclude alternative (a), for instance by proving the spider inequality of the remark after Theorem 1.5 and completing the replacement argument sketched there. This would make the core 2-connected.

2. Exclude alternative (b): show that, in a minimal counterexample, a hub adjacent to a decorated vertex forces $\mathrm{drop}(L) \ge 1$. Simulated annealing never produced a hub adjacent to a support together with $\mathrm{drop}(L) < 1$.

References

[1] B. Bollobás and P. Erdős, Graphs of extremal weights, *Ars Combin.* **50** (1998) 225–233.

[2] G. Caporossi and P. Hansen, Variable neighborhood search for extremal graphs. 1. The AutoGraphiX system, *Discrete Math.* **212** (2000) 29–44.

[3] M. Cygan, M. Pilipczuk and R. Škrekovski, On the inequality between radius and Randić index for graphs, *MATCH Commun. Math. Comput. Chem.* **67**(2) (2012) 451–466.

[4] H. Deng, Z. Tang and J. Zhang, On a conjecture of Randić index and graph radius, *Filomat* **29** (2015); arXiv:1210.2543.

[5] M. I. Doig, Randić index, radius, and diameter for cactus graphs, arXiv:2107.00071.

[6] M. I. Doig, Block eccentricity, a radius bound, and an application to the Randić index, arXiv:2309.11613.

[7] Z. Dvořák, B. Lidický and R. Škrekovski, Randić index and the diameter of a graph, *European J. Combin.* **32** (2011) 434–442.

[8] S. Fajtlowicz, On conjectures of Graffiti, *Discrete Math.* **72** (1988) 113–118.

[9] B. Liu and I. Gutman, paper on the Randić index and the radius (2009), as cited in [7, 4, 5]; not consulted by us.

[10] B. D. McKay and A. Piperno, Practical graph isomorphism, II, *J. Symbolic Comput.* **60** (2014) 94–112.

[11] M. Randić, On characterization of molecular branching, *J. Amer. Chem. Soc.* **97** (1975) 6609–6615.

[12] Y. Yang and L. Lu, The Randić index and the diameter of graphs, *Discrete Math.* (2011); arXiv:1104.0426.

[13] Z. You and B. Liu, paper on the Randić index and the radius (2009), as cited in [7, 4, 5]; not consulted by us.

A Code

A.1 `bmu.c`: the finite verification for Theorem 1.4

```
// Bounded-cyclomatic verification for the Caporossi-Hansen conjecture.
// Reads cores K (graph6, connected, min degree >=2) from stdin. For each K that is bridgeless with at most
     3
// vertices of degree 2, enumerates decorated graphs G = K + (one leaf at each vertex of S) + (optional
    pendant path of
// length s>=2 at w, 2<=s<=ecc_K(w)), with every degree-2 vertex of K in S or equal to w, and with drop(L)
    <1
// (L = leaves of G). For those satisfying E2 (every K-edge e has R(G-e) > R(G)), checks R(G) >= rad(G).
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdint.h>
#define MAXN 64
typedef uint64_t u64;
static int n; static u64 A[MAXN];
static int nk; static u64 KA[32];
static int popc(u64 x){return __builtin_popcountll(x);}
static void parse_g6(const char*s){ int k=0; nk=s[k++]-63; memset(KA,0,sizeof KA); int bits=0; unsigned
    char c=0;
  for(int j=1;j<nk;j++) for(int i=0;i<j;i++){ if(bits==0){c=s[k++]-63; bits=6;} bits--; if((c>>bits)&1){KA
    [i]|=1ULL<<j; KA[j]|=1ULL<<i;} } }
static int eccM(u64 *M,u64 all,int s){ u64 seen=1ULL<<s, fr=seen; int d=0; while(1){ u64 nf=0; for(u64 t=
    fr;t;t&=t-1) nf|=M[__builtin_ctzll(t)]; nf&=all&~seen; if(!nf) break; d++; seen|=nf; fr=nf;} if(seen
    !=all) return 999; return d; }
static int radM(u64 *M,u64 all){ int r=999; for(int v=0;v<64;v++) if((all>>v)&1){int x=eccM(M,all,v); if(x
    <r) r=x;} return r; }
```

```
static double RM(u64 *M,u64 all){ double R=0; int deg[MAXN]; for(int v=0;v<n;v++) deg[v]=popc(M[v]&all);
     for(int u=0;u<n;u++) if((all>>u)&1) for(u64 t=M[u]&all;t;t&=t-1){int v=__builtin_ctzll(t); if(v>u) R
     +=1.0/sqrt((double)deg[u]*deg[v]);} return R; }
static int kbridgeless(){ u64 all=(1ULL<<nk)-1; for(int u=0;u<nk;u++) for(u64 t=KA[u];t;t&=t-1){int v=
     __builtin_ctzll(t); if(v<u) continue; KA[u]&=~(1ULL<<v); KA[v]&=~(1ULL<<u); int ok=eccM(KA,all,0)
     !=999; KA[u]|=1ULL<<v; KA[v]|=1ULL<<u; if(!ok) return 0;} return 1; }
static int W,Slen; static u64 Smask;
static void build(){ n=nk; memset(A,0,sizeof A); for(int u=0;u<nk;u++) A[u]=KA[u];
  for(int u=0;u<nk;u++) if((Smask>>u)&1){ A[u]|=1ULL<<n; A[n]=1ULL<<u; n++; }
  if(W>=0){ int prev=W; for(int i=0;i<Slen;i++){ A[prev]|=1ULL<<n; A[n]|=1ULL<<prev; prev=n; n++; } } }
static double dRe(int u,int v){ int du=popc(A[u]), dv=popc(A[v]); double fu=1/sqrt(du), fv=1/sqrt(dv);
  double Su=0,Sv=0; for(u64 t=A[u]&~(1ULL<<v);t;t&=t-1){int x=__builtin_ctzll(t); Su+=1/sqrt(popc(A[x]));}
  for(u64 t=A[v]&~(1ULL<<u);t;t&=t-1){int x=__builtin_ctzll(t); Sv+=1/sqrt(popc(A[x]));}
  double pu=(du>1)?(1/sqrt(du-1.0)-fu):0, pv=(dv>1)?(1/sqrt(dv-1.0)-fv):0; return -fu*fv+pu*Su+pv*Sv; }
static long nK=0, nconf=0, ne2=0, nviol=0; static double minslack=1e9; static char worstbuf[512];
static char curg6[128];
static double dropL(){ u64 all=(1ULL<<n)-1, L=0; for(int x=0;x<n;x++) if(popc(A[x]&all)==1) L|=1ULL<<x;
     return RM(A,all)-RM(A,all&~L); }
static void check(){ nconf++;
  // E2 on K-edges (K-edges are exactly the non-bridge edges of G)
  for(int u=0;u<nk;u++) for(u64 t=KA[u];t;t&=t-1){int v=__builtin_ctzll(t); if(v<u) continue; if(dRe(u,v)<
      -1e-9) return;}
  ne2++;
  u64 all=(1ULL<<n)-1; double R=RM(A,all); int r=radM(A,all); double s=R-r;
  if(s<minslack){ minslack=s; snprintf(worstbuf,sizeof worstbuf,"K=%s S=%llx w=%d s=%d n=%d r=%d R=%.5f",
     curg6,(unsigned long long)Smask,W,Slen,n,r,R); }
  if(s<1e-9){ nviol++; printf("VIOLATION K=%s S=%llx w=%d s=%d n=%d r=%d R=%.6f\n",curg6,(unsigned long
     long)Smask,W,Slen,n,r,R); fflush(stdout); } }
static u64 forced; static int order[32];
static void dfs(int idx){ // decide vertices order[idx..] ; Smask holds current supports
  build(); double d=dropL(); if(d>=1.0+1e-9) return; // monotone: supersets also have drop>=1
  if(idx==nk){ if((forced&~Smask)==0) check(); return; }
  int v=order[idx];
  // option: not a support (only allowed if not forced)
  if(!((forced>>v)&1)) dfs(idx+1);
  Smask|=1ULL<<v; dfs(idx+1); Smask&=~(1ULL<<v); }
int main(){ char line[256];
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse_g6(line); strcpy(curg6,line);
    int d2=0; u64 d2m=0; for(int v=0;v<nk;v++){ int d=popc(KA[v]); if(d<2) goto next; if(d==2){d2++; d2m
     |=1ULL<<v;} }
    if(d2>3) continue; if(!kbridgeless()) continue; nK++;
    for(int i=0;i<nk;i++) order[i]=i;
    // no long path
    W=-1; Slen=0; Smask=0; forced=d2m; dfs(0);
    for(int w=0;w<nk;w++){ int ew=eccM(KA,(1ULL<<nk)-1,w); for(int s=2;s<=ew;s++){ W=w; Slen=s; Smask=0;
     forced=d2m&~(1ULL<<w); dfs(0); } }
    next:;
  }
  printf("cores %ld configs(drop<1) %ld E2-irreducible %ld violations %ld minslack %.6f at %s\n",nK,nconf,
     ne2,nviol,minslack,worstbuf); }
```

A.2 `run_cyclomatic.sh`: driver

```
#!/bin/bash
# Driver for the bounded-cyclomatic-number verification: for each cyclomatic number mu<=MU and each core
     order nk<=2mu+1,
# enumerate cores with nauty geng (connected, min degree >= 2, exactly nk+mu-1 edges) and run bmu.
# The bound -D(2mu+4-nk) on the maximum degree only speeds up geng: every core kept by bmu's filter
# (at most 3 vertices of degree 2, all others of degree >= 3, degree sum 2(nk+mu-1)) satisfies it.
# usage: ./run_cyclomatic.sh MU PARTS
GENG=geng   # path to nauty's geng
MU=$1; P=${2:-4}
for mu in $(seq 1 $MU); do for nk in $(seq 3 $((2*mu+1))); do
  e=$((nk+mu-1)); lo=$(( (3*nk-3+1)/2 )); maxe=$(( nk*(nk-1)/2 )); D=$((2*mu+4-nk))
  if [ $e -ge $lo ] && [ $e -le $maxe ]; then
    for ((p=0;p<P;p++)); do ( $GENG -cq -d2 -D$D $nk $e:$e $p/$P | ./bmu > out_nk${nk}_mu${mu}_p${p}.txt )
      & done; wait
  fi
done; done
cat out_* | awk '{c+=$2; cf+=$4; e2+=$6; v+=$8} END {print "cores",c,"configs",cf,"E2-irreducible",e2,"
     violations",v}'
```

A.3 `xcheck.py`: independent brute-force cross-check

```
# Independent brute-force cross-check of bmu.c for small cyclomatic number:
# no pruning, no edge-deletion formula; every quantity recomputed from scratch.
import sys, math, subprocess
GENG='geng'   # path to nauty's geng
def g6(s):
    n=ord(s[0])-63; bits=[]
    for ch in s[1:]:
        v=ord(ch)-63; bits+=[(v>>(5-i))&1 for i in range(6)]
    adj=[set() for _ in range(n)]; k=0
    for j in range(1,n):
        for i in range(j):
            if bits[k]: adj[i].add(j); adj[j].add(i)
            k+=1
    return adj
def dists(adj,s,alive):
    d={s:0}; fr=[s]
    while fr:
        nf=[]
        for x in fr:
            for y in adj[x]:
                if y in alive and y not in d: d[y]=d[x]+1; nf.append(y)
        fr=nf
    return d
def connected(adj,alive):
    s=next(iter(alive)); return len(dists(adj,s,alive))==len(alive)
def R(adj,alive):
    deg={v:sum(1 for y in adj[v] if y in alive) for v in alive}
    return sum(1/math.sqrt(deg[a]*deg[b]) for a in alive for b in adj[a] if b in alive and a<b)
def rad(adj,alive):
    return min(max(dists(adj,v,alive).values()) for v in alive)
def bridgeless(adj):
    n=len(adj); alive=set(range(n))
    for a in range(n):
        for b in list(adj[a]):
            if a<b:
                adj[a].discard(b); adj[b].discard(a)
                ok=connected(adj,alive)
                adj[a].add(b); adj[b].add(a)
                if not ok: return False
    return True
tot={}
MU=int(sys.argv[1])
for mu in range(1,MU+1):
    cores=confs=irr=viol=0
    for nk in range(3,2*mu+2):
        e=nk+mu-1
        if e<(3*nk-3+1)//2 or e>nk*(nk-1)//2: continue
        out=subprocess.run([GENG,'-cq','-d2',str(nk),f'{e}:{e}'],capture_output=True,text=True).stdout.
     split()
        for line in out:
            K=g6(line)
            d2=[v for v in range(nk) if len(K[v])==2]
            if len(d2)>3 or not bridgeless(K): continue
            cores+=1
            eccK=[max(dists(K,v,set(range(nk))).values()) for v in range(nk)]
            opts=[(None,0)]+[(w,s) for w in range(nk) for s in range(2,eccK[w]+1)]
            for (w,s) in opts:
                for mask in range(1<<nk):
                    U=[v for v in range(nk) if (mask>>v)&1]
                    if any((v not in U) and v!=w for v in d2): continue
                    adj=[set(x) for x in K]
                    def add(a):
                        adj.append(set()); b=len(adj)-1; adj[a].add(b); adj[b].add(a); return b
                    for u in U: add(u)
                    if w is not None:
                        p=w
                        for _ in range(s): p=add(p)
                    alive=set(range(len(adj)))
                    L={v for v in alive if len(adj[v])==1}
                    RG=R(adj,alive)
                    drop=RG-R(adj,alive-L)
```

```
                    if drop>=1+1e-9: continue
                    confs+=1
                    ok=True
                    for a in range(nk):
                        for b in K[a]:
                            if a<b:
                                adj[a].discard(b); adj[b].discard(a)
                                dR=R(adj,alive)-RG
                                adj[a].add(b); adj[b].add(a)
                                if dR< -1e-9: ok=False; break
                        if not ok: break
                    if not ok: continue
                    irr+=1
                    r=rad(adj,alive)
                    if RG-r<1e-9: viol+=1; print("VIOLATION",line,U,w,s,RG,r)
    print(f"mu={mu}: cores {cores} configs {confs} E2-irreducible {irr} violations {viol}",flush=True)
```

A.4 `peel10.c`: Peeling Conjecture for $n \leq 10$

```
// For E2-irreducible non-tree graphs: compute drop(L) = R(G) - R(G - all leaves), rad(G-L), and flag
    failures.
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#define MAXN 32
static int n; static unsigned adj[MAXN];
static int popc(unsigned x){return __builtin_popcount(x);}
static void parse_g6(const char*s){ int k=0; n=s[k++]-63; memset(adj,0,sizeof adj); int bits=0; unsigned
    char c=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){ if(bits==0){c=s[k++]-63; bits=6;} bits--; if((c>>bits)&1){adj
    [i]|=1u<<j; adj[j]|=1u<<i;} } }
static int eccA(unsigned *A,unsigned all,int s){ unsigned seen=1u<<s, fr=seen; int d=0; while(1){ unsigned
    nf=0; for(unsigned t=fr;t;t&=t-1) nf|=A[__builtin_ctz(t)]; nf&=all&~seen; if(!nf) break; d++; seen|=nf
    ; fr=nf;} if(seen!=all) return 999; return d; }
static int radA(unsigned *A,unsigned all){ int r=999; for(int v=0;v<n;v++) if((all>>v)&1){int x=eccA(A,all,
    v); if(x<r) r=x;} return r; }
static double RA(unsigned *A,unsigned all){ int deg[MAXN]; for(int v=0;v<n;v++) deg[v]=popc(A[v]&all);
    double R=0; for(int u=0;u<n;u++) if((all>>u)&1) for(unsigned t=A[u]&all;t;t&=t-1){int v=__builtin_ctz(
    t); if(v>u) R+=1.0/sqrt((double)deg[u]*deg[v]);} return R; }
static int isevenpath(unsigned *A,unsigned all){ int c=popc(all); if(c%2) return 0; int e=0,c1=0,c2=0; for(
    int v=0;v<n;v++) if((all>>v)&1){int d=popc(A[v]&all); e+=d; if(d==1)c1++; else if(d==2)c2++; else
    return 0;} e/=2; return (e==c-1 && (c==2? c1==2 : (c1==2 && c2==c-2))); }
int main(int argc,char**argv){ char line[256]; long nirr=0, bad=0; double mind=1e9; char mg[64]="";
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse_g6(line); unsigned all=(1u<<n)-1;

    int e=0; for(int v=0;v<n;v++) e+=popc(adj[v]); e/=2; if(e==n-1) continue;
    double R=RA(adj,all); int irr=1;
    for(int u=0;u<n && irr;u++) for(unsigned t=adj[u];t && irr;t&=t-1){ int v=__builtin_ctz(t); if(v<u)
     continue;
      unsigned B[MAXN]; memcpy(B,adj,sizeof B); B[u]&=~(1u<<v); B[v]&=~(1u<<u);
      if(eccA(B,all,0)==999) continue;
      if(RA(B,all)<R-1e-9) irr=0; }
    if(!irr) continue; nirr++;
    unsigned L=0; for(int v=0;v<n;v++) if(popc(adj[v])==1) L|=1u<<v;
    unsigned rest=all&~L; int r=radA(adj,all); int r2=radA(adj,rest); double R2=RA(adj,rest); double drop=
     R-R2;
    int ep=isevenpath(adj,rest);
    double need = (r - r2) + (ep?0.0858:0); // need drop >= r - r2 (+ evenpath deficit)
    if(drop < mind){mind=drop; strcpy(mg,line);}
    if(drop < need + 1e-9){ bad++; printf("FAIL %s n=%d r=%d r(G-L)=%d drop=%.4f leaves=%d evenpath=%d\n",
     line,n,r,r2,drop,popc(L),ep); }
  }
  printf("E2-irreducible %ld ; leaf-peeling fails %ld ; min drop %.4f at %s\n",nirr,bad,mind,mg); }
```

A.5 `idcheck.py`, `brcheck.py`, `pathb.py`: sanity checks

```
# Numerical sanity checks of Lemmas 2.1, 2.4, 2.6 and 2.7 on random graphs.
```

```
import networkx as nx, random, math
def R(G):
    d=dict(G.degree()); return sum(1/math.sqrt(d[u]*d[v]) for u,v in G.edges())
def psi(d): return 0.0 if d<=1 else math.sqrt(d*(d-1))-(d-1)
random.seed(7); bad=0; tested=0
for it in range(3000):
    n=random.randint(4,14); p=random.choice([0.15,0.25,0.35,0.5])
    G=nx.gnp_random_graph(n,p,seed=random.randint(0,10**9))
    if not nx.is_connected(G) or G.number_of_edges()<2: continue
    d=dict(G.degree()); f={v:1/math.sqrt(d[v]) for v in G}
    # (1) edge deletion formula Delta R(uv) = f_u f_v (psi_u A_u(v) + psi_v A_v(u) - 1)
    for u,v in G.edges():
        H=G.copy(); H.remove_edge(u,v); actual=R(H)-R(G)
        def A(x,y):
            if d[x]==1: return 0.0
            return sum(math.sqrt(d[y]/d[z]) for z in G[x] if z!=y)/(d[x]-1)
        formula=f[u]*f[v]*(psi(d[u])*A(u,v)+psi(d[v])*A(v,u)-1)
        tested+=1
        if abs(actual-formula)>1e-9: bad+=1
    # (2) radius-stable peeling: S with each s in S having a neighbour outside S and G-S connected -> rad(G-
    S)>=rad(G)-1
    S=set(v for v in G if random.random()<0.3)
    if S and len(S)<n-1:
        H=G.copy(); H.remove_nodes_from(S)
        if nx.is_connected(H) and all(any(w not in S for w in G[s]) for s in S):
            tested+=1
            if nx.radius(H) < nx.radius(G)-1: bad+=1; print("peeling counterexample",G.edges(),S)
    # (3) leaf removal strictly decreases R
    for l in [v for v in G if d[v]==1]:
        H=G.copy(); H.remove_node(l)
        if H.number_of_nodes()>=2:
            tested+=1
            a=next(iter(G[l]))
            if not (R(G)-R(H) >= math.sqrt(d[a])-math.sqrt(d[a]-1)-1e-12): bad+=1; print("leaf drop fail")
    # (4) vertex deletion formula (Lemma 2.4) for a vertex b whose neighbours all have degree >= 2
    for b in G:
        if all(d[a]>=2 for a in G[b]) and G.number_of_nodes()>=3:
            H=G.copy(); H.remove_node(b); actual=R(G)-R(H)
            ph={a:1/math.sqrt(d[a]-1)-f[a] for a in G[b]}
            formula=sum(f[b]*f[a]-ph[a]*sum(f[y] for y in G[a] if y!=b) for a in G[b])
            formula-=sum(ph[a]*ph[c] for a in G[b] for c in G[b] if a<c and G.has_edge(a,c))
            tested+=1
            if abs(actual-formula)>1e-9: bad+=1; print("vertex formula fail")
    # (5) leaf-set peeling formula and monotonicity (Lemma 2.7): attach leaves at a random set U of
    vertices
    U=[v for v in G if random.random()<0.5]
    if U:
        G2=G.copy(); lv={}
        for u in U: lv[u]=('leaf',u); G2.add_edge(u,lv[u])
        d2=dict(G2.degree()); f2={v:1/math.sqrt(d2[v]) for v in G2}; ph2={u:1/math.sqrt(d2[u]-1)-f2[u] for
     u in U}
        actual=R(G2)-R(G)
        t={u:f2[u]-ph2[u]*sum(f2[y] for y in G2[u] if y!=lv[u]) for u in U}
        formula=sum(t[u]-0.5*ph2[u]*sum(ph2[v] for v in G2[u] if v in ph2) for u in U)
        tested+=1
        if abs(actual-formula)>1e-9 or actual<=0: bad+=1; print("drop formula fail")
        rest=[v for v in G if v not in U]
        if rest:
            x=random.choice(rest); G3=G2.copy(); G3.add_edge(x,('leaf',x))
            tested+=1
            if not (R(G3)-R(G) > actual): bad+=1; print("drop monotonicity fail")
print("tested",tested,"bad",bad)
```

```
# Check the bridge replacement lemma (BR): for a bridge xy with e_x >= e_y and d_G(x)>=2, replacing G_y by
    a path
# of length e_y+1 at x gives G' with R(G') <= R(G) and rad(G') >= rad(G).  Also check the rooted Yang-Lu
    lemma (RL).
import networkx as nx, random, math
def R(G):
    d=dict(G.degree()); return sum(1/math.sqrt(d[u]*d[v]) for u,v in G.edges())
random.seed(11); tested=0; bad=0; rlbad=0; rltested=0
for it in range(6000):
    # build: random connected graph + random pendant structures to create bridges
```

```
    n=random.randint(3,10); G=nx.gnp_random_graph(n,random.choice([0.3,0.45,0.6]),seed=random.randint
     (0,10**9))
    if not nx.is_connected(G): continue
    m=n
    for k in range(random.randint(1,3)):
        # attach a random small connected graph via a bridge
        h=random.randint(1,6); H=nx.gnp_random_graph(h,random.choice([0.3,0.6,0.9]),seed=random.randint
     (0,10**9))
        if not nx.is_connected(H): continue
        H=nx.relabel_nodes(H,{i:m+i for i in range(h)}); G=nx.union(G,H); G.add_edge(random.randrange(m),m
     +random.randrange(h)); m+=h
    G=nx.convert_node_labels_to_integers(G)
    for (x,y) in list(nx.bridges(G)):
        for (a,b) in [(x,y),(y,x)]:
            H=G.copy(); H.remove_edge(a,b)
            Ca=nx.node_connected_component(H,a); Cb=nx.node_connected_component(H,b)
            ea=max(nx.single_source_shortest_path_length(H.subgraph(Ca),a).values())
            eb=max(nx.single_source_shortest_path_length(H.subgraph(Cb),b).values())
            if ea<eb or G.degree(a)<2: continue
            # RL check on (G_b, b): R~{+b}(G_b) >= eb/2 + (1-1/sqrt(d+1))/sqrt(2)
            Gb=H.subgraph(Cb).copy(); db=Gb.degree(b)
            if Gb.number_of_nodes()>=2:
                dd=dict(Gb.degree()); dd[b]+=1
                Rp=sum(1/math.sqrt(dd[u]*dd[v]) for u,v in Gb.edges())
                rltested+=1
                if Rp < eb/2 + (1-1/math.sqrt(db+1))/math.sqrt(2) - 1e-9: rlbad+=1
            if len(Cb)==eb+1: continue  # already a path
            G2=G.copy(); G2.remove_nodes_from(Cb); prev=a; nxt=max(G2.nodes())+1
            for i in range(eb+1): G2.add_edge(prev,nxt); prev=nxt; nxt+=1
            tested+=1
            if R(G2) > R(G)+1e-9 or nx.radius(G2) < nx.radius(G): bad+=1; print("BR fail")
print("BR tested",tested,"bad",bad,"; RL tested",rltested,"bad",rlbad)
```

```
# Even-path case in the proof of Lemma 6.2: G = P_{2k} + b with b adjacent to 3 consecutive path vertices;
     check R(G) - k and R(G)-R(G-b).
import math
def R_of(edges):
    from collections import Counter
    d=Counter()
    for u,v in edges: d[u]+=1; d[v]+=1
    return sum(1/math.sqrt(d[u]*d[v]) for u,v in edges)
worst=1e9; worstD=1e9
for k in range(2,40):
    P=[(i,i+1) for i in range(1,2*k)]
    for i in range(1,2*k-1):
        E=P+[('b',i),('b',i+1),('b',i+2)]
        Rg=R_of(E); Rp=R_of(P)
        worst=min(worst,Rg-k); worstD=min(worstD,Rg-Rp)
print("min R(G)-k =",worst," min R(G)-R(G-b) =",worstD)
```